Billions for nothing
Nestlé makes billions bottling water it pays nearly nothing for from economically depressed areas with lax water laws. http://dgit.in/WaterMoney

New iPhones top DxOMark
Apple's newly launched iPhone 8 and 8 Plus cameras get top marks from testing outfit DxOMark. http://dgit.in/iPhone8DxO

Here we have declared the default value of b as '-1'.

In this case, we can call the sum function by a simgle parameter instead of two parameters.

So the complete code for sum will look like this:

```
fun main(args: Array<String>)
{
    println("The sum of 1 and
2 is ${sum(1, 2)}")
    println("Reducing 1 from 3
will give us ${reduce1(3)}")
}
fun sum(a: Int, b: Int = -1) =
a + b
fun reduce1(a: Int) = sum(a)
```

This code will generate this output:

```
/usr/local/java/jdk1.8.0_111/bin/java ...
The sum of 1 and 2 is 3
Reducing 1 from 3 will give us 2

Process finished with exit code 0
```

3. DEFINING LOCAL VARIABLES:

There are two types of variable that you can create in Kotlin. One is Type defined or Type inferred. Note that if you don't want to define the type of the variable then you have to assign some value to that variable so that the kotlin compiler gets to know about the type of value that the variable will use.

Another aspect of Kotlin is you can create Assign-Once variables where the variable behaves as read-only. Or you can create mutable variables where you can manipulate the value of the variable. Here are some examples to help you out.

Read Only Variables:

```
val a: Int = 1  // immediate
assignment
val b = 2   // `Int` type is
inferred
val c: Int  // Type required
when no initializer is pro-
vided
c = 3       // deferred as-
signment
```

Mutable Variables

```
var x = 5 // `Int` type is
inferred
x += 1
```

3. DECLARING CLASSES

Classes in Kotlin are declared with the keyword Class. To define a basic class you use the following syntax:

```
class Invoice {
}
```

The class declaration consists of the class name, the class header (specifying its type parameters, the primary constructor etc.) and the class body, surrounded by curly braces. Both the header and the body are optional; if the class has no body, curly braces can be omitted. Example:

```
class Empty
```

Constructors

A class in Kotlin can have a primary constructor and one or more secondary constructors. The primary constructor is part of the class header: it goes after the class name (and optional type parameters).

```
class Person
constructor(firstName: String)
{}
```

If the primary constructor does not have any annotations or visibility modifiers, the constructor keyword can be omitted:

```
class Person(firstName:
String) {
}
```

The primary constructor cannot contain any code. Initialization code can be placed in initializer blocks, which are tp be prefixed with the 'init' keyword:

```
class Customer(name: String) {
    init {
        println("Customer ini-
tialized with value ${name}")
    }
}
```

Note that parameters of the primary constructor can be used in the initializer blocks. They can also be used in property initializers declared in the class body:

```
class Customer(name: String) {
    val customerKey = name.
toUpperCase()
}
```

In fact, for declaring properties and initializing them from the primary constructor, Kotlin has a concise syntax to be used:

```
class Person(val firstName:
String, val lastName: String,
var age: Int) {
    // ...
}
```

Much the same way as regular properties, the properties declared in the primary constructor can be mutable (var) or read-only (val).

If the constructor has annotations or visibility modifiers, the constructor keyword is required, and the modifiers go before it:

```
class Customer public @Inject
constructor(name: String) {
... }
```

Just Like Java Access Specifiers Kotlin has visibility modifiers. They are

- private — visible inside this class only (including all its members);

- protected — same as private + visible in subclasses too;

- internal — any client inside this module who sees the declaring class can see its internal members;

- public — any client who sees the declaring class can see its public members.

NOTE for Java users: outer class does not see private members of its inner classes in Kotlin.

Creating instances of Classes

```
val invoice = Invoice()
val customer = Customer("Joe
Smith")
```

Note that Kotlin does not have a new keyword.

Creating instances of nested, inner and anonymous inner classes is described in Nested classes.

4. JAVA-KOTLIN ITERPORTABILITY

Calling Java from Kotlin

Kotlin is designed with Java Interoperability in mind. Existing Java code can be called from Kotlin in a natural way, and Kotlin code can be used from Java rather smoothly as well. In this section we describe some of the details regarding calling Java code from Kotlin.

Pretty much all Java code can be used without any issues: